

The Same Agent in StateTree

The StateTree route to identical flight behaviour.

~15 minutes. Do Tutorial 03 first. This chapter only covers the differences

IN SHORT

Identical flight behaviour to [Tutorial 03](#), authored in StateTree instead of a Behavior Tree. Swap the AIController's `RunBehaviorTree` for a `StateTreeAIComponent`, and bind the `Fly To` task's `TargetLocation` instead of a blackboard key.

Should you use StateTree or Behavior Tree?

Both are fully supported and produce the same flight. Pick on authoring preference, not capability.

	Behavior Tree	StateTree
Goal comes from	A blackboard key	Bound instance data on the task
Best at	Reactive priority ordering	Explicit states and transitions
Maturity on UE 5.8	Stable	Stable (Epic's StateTree plugin left experimental at 5.3)

NOTE

No maturity caveat on this engine version. Earlier LCM Nav3D builds were flagged beta on UE 5.2, because Epic's StateTree plugin was itself experimental there. StateTree has been production stable since 5.3, so both routes are equally safe to ship and the choice is purely about which authoring model you prefer.

Step 1: The pawn

Exactly as [Tutorial 03 Step 2](#): a Pawn with **Floating Pawn Movement**, a visible mesh, and **Auto Possess AI = Placed in World or Spawned**.

Nothing StateTree-specific here.

Step 2: Create the StateTree asset

1. Content Browser → Add → Artificial Intelligence → StateTree.
2. When it asks for a schema, choose **StateTree AI Component Schema**. This matters: the AI schema is what exposes the AI context (controller and pawn) the Fly To task needs.
3. Name it `ST_MyFlyer`.

Step 3: Add the Fly To task

1. Open `ST_MyFlyer`. You'll have a **Root** state.
2. Add a child state and name it `FlyToGoal`.
3. With that state selected, in the **Tasks** section click + and choose **Fly To** (listed under the **LCM Nav3D** category).
4. Set the task's properties:

Field	Value	Notes
Target Location	your goal	See binding below
Acceptance Radius	<code>100</code>	
Flight Speed	<code>600</code>	Match your Floating Pawn Movement <code>Max Speed</code>

Binding the goal

This is *the* structural difference from Behavior Trees. There's no blackboard; you bind the task's `Target Location` to a value.

The simplest approach is a **StateTree** parameter:

1. Select the **Root** state → in **Parameters**, add a parameter: Name `GoalLocation`, Type `Vector`.
2. Select the **Fly To** task → click the bind icon next to **Target Location** → choose `GoalLocation`.

Now anything that sets that parameter drives the agent.

NOTE

Chasing instead. The task also has a `Target Actor` field. Set or bind that and the agent chases the actor, using the same prediction logic as the BT version. Note the ST chase path requires `Target Actor` to be non-null. If you want a fixed point, use `Target Location`.

Step 4: The AI Controller

1. Create an AIController Blueprint, `BP_MyFlyerAI_ST`.
2. Add a StateTree AI Component.
3. Select it and set **State Tree** to `ST_MyFlyer`.
4. Set the goal parameter on BeginPlay. On the component, use **Set Parameter** (or set the value before the tree starts) with:
 - **Name:** `GoalLocation`
 - **Value:** e.g. `GetActorOfClass(TargetPoint) → GetActorLocation`
5. On `BP_MyFlyer` → Class Defaults → AI Controller Class = `BP_MyFlyerAI_ST`.

NOTE

Unlike the BT path there is no `Run Behavior Tree` call: the `StateTreeAIComponent` starts the tree itself.

Step 5: Fly

Place a `TargetPoint` and your pawn, press **Play**. Same result as Tutorial 03: the pawn routes through 3D space to the goal.

The rest of the StateTree surface

The plugin ships a full StateTree task set, all under the **LCM Nav3D** category:

Task	Does
Fly To	The flagship: 3D navigation to a point or actor
Patrol	Cycle through a set of points
Orbit Target	Circle a target at a radius
Land	Descend and touch down
Look At / Turn In Place	Orientation without translation
Find Cover	Move to tactical cover
Evade CBS	Avoidance-driven evasion
Follow Flow Field	Follow a shared crowd steering field
Chunk Prewarm / Streaming Aware	Infinite-world streaming helpers
Find Actor By Tag / Pick Random Goal / Wait	Utility
Use Smart Object	Smart Object interaction

Plus conditions (`Path Exists` , `Is At Location` , `Chunk Loaded` , `Has Line Of Sight` , `Is In Cover` , `Health Below` , `Path Length Below` , `Threat Visible Via Perception`) and evaluators (`Path Status` , `Path Progress` , `Navigation Load` , `Threat Field` , `Squad` , `Memory` , `Perception Relay`).

Every one is listed with its full property set in the [API Reference](#).

Behaviour differences worth knowing

The BT and ST Fly To tasks are built for parity, but a few things genuinely differ:

	Behavior Tree	StateTree
Goal source	Blackboard key (Vector or Actor)	<code>Target Location</code> or <code>Target Actor</code> instance data
Chase	Works with a Vector or Actor key	Requires a non-null <code>Target Actor</code>
Arrival reported	Against the pre-move location that frame	Against the post-move location

The arrival difference is a sub-frame detail: it only matters if you're comparing the two side-by-side with a very small `Acceptance Radius` .

Troubleshooting

Symptom	Cause
Fly To isn't in the task list	Asset wasn't created with the StateTree AI Component Schema
Task fails immediately	No LcmNavigationManagerSV0 in the level, or the goal is outside/inside geometry
Agent never starts	StateTreeAIComponent has no State Tree assigned, or Auto Possess AI is off
Goal is always (0,0,0)	The parameter isn't bound, or is set after the tree already ran

More: [Troubleshooting](#).

NEXT

[Tutorial 05: Dynamic Obstacles](#). Make the world move and watch agents reroute.

Chapter 4 of the LCM Nav3D User Manual.
The complete manual is **LCM_Nav3D_Manual.pdf** , in this folder alongside every other document in the set. Start from **README.pdf** .